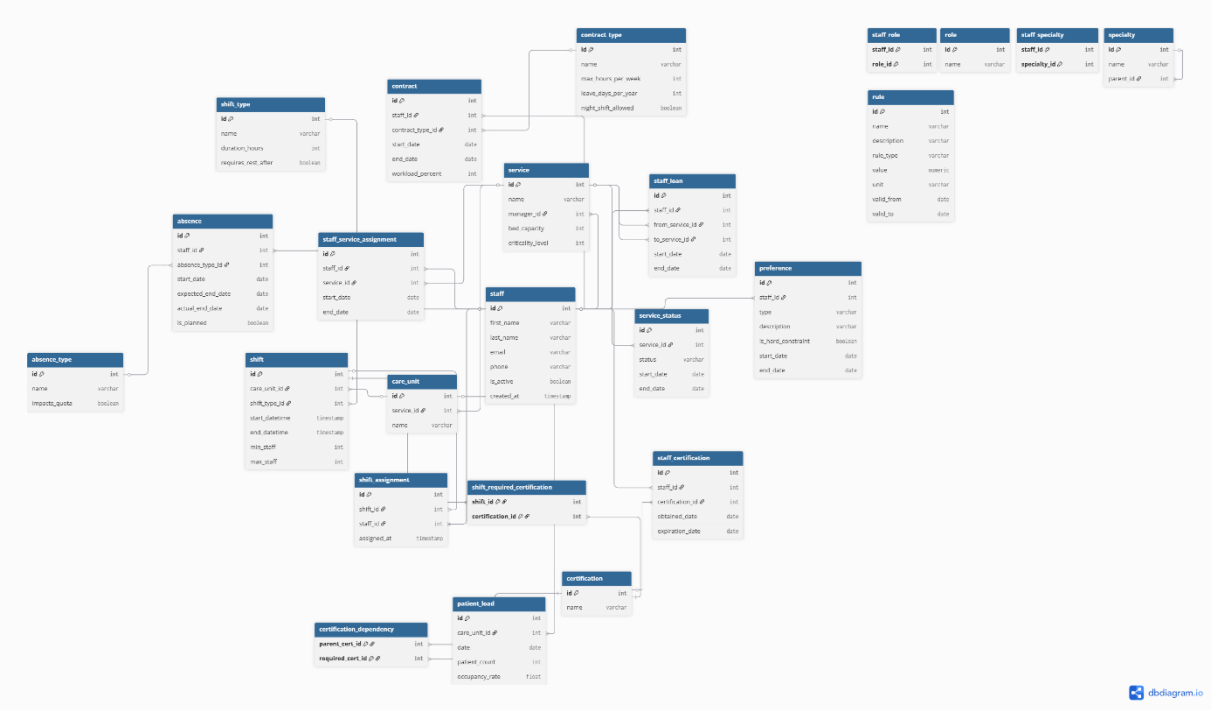
PHASE 01

Fondations – Modélisation & Base de Données

Du domaine métier au schéma relationnel normalisé

SQL & Modélisation

L'objectif de ce document est d'analyser une proposition d'une architecture de la base de données HospiPlan (Phase 1 – Bootcamp) accessible via ce lien : https://dbdiagram.io/d/Version-1-69c2eebd78c6c4bc7a5bab3a



Analyse :

Module 1 : Identité, Compétences et Hiérarchie (F-01, F-03)

Les Contraintes :

- Distinguer les catégories (médecins, infirmiers) et spécialités.
- Gérer une hiérarchie de spécialités (ex: la cardiologie pédiatrique est une sous-spécialité de la pédiatrie).
- Tracer les certifications avec prérequis et dates d'expiration.

La Solution Architecturale :

- **Polymorphisme des rôles :** Au lieu d'utiliser des tables séparées pour chaque métier, nous utilisons une table staff centrale liée à une table role via une table de liaison staff_role. Cela permet à un agent d'évoluer ou de cumuler des fonctions.

- **Récursivité (Adjacency List) :** Dans la table specialty, l'attribut parent_id référence la clé primaire id de la même table. Cela permet de construire un arbre de spécialités sans profondeur limite.
- **Graphe de Certifications :** La table certification_dependency permet de définir qu'une certification (ex: Réanimation avancée) ne peut être obtenue sans une autre (ex: Secourisme de base). L'intégrité temporelle est assurée dans staff_certification par l'attribut expiration_date.

Module 2 : Temporalité des Contrats et Absences (F-02, F-06)

Les Contraintes :

- Gérer les types de contrats (CDI, CDD, Intérim) et leurs quotas d'heures.
- Maintenir un historique des affectations.
- Distinguer les absences prévues des imprévues.

La Solution Architecturale :

- **Historisation par intervalles :** Les tables contract et absence n'écrasent jamais les données. Elles utilisent un couple start_date / end_date.
    - *Astuce de modélisation :* Un end_date à NULL dans la table contract signifie que le contrat est toujours actif.
- **Logique d'absence :** La table absence comporte trois colonnes temporelles : start_date, expected_end_date (fin prévue) et actual_end_date (fin réelle). Cette structure permet de calculer l'écart entre le planning théorique et la réalité de l'unité de soin.

Module 3 : Infrastructure et Charge Dynamique (F-04, F-08, F-09)

Les Contraintes :

- Structurer les services et les unités de soins.
- Suivre le taux d'occupation quotidien des lits.
- Tracer les "prêts" de personnel entre services sans rompre l'affectation d'origine.

La Solution Architecturale :

- **Granularité de l'organisation :** La relation entre service et care_unit est une relation **1:N**. Un service peut ainsi posséder plusieurs unités (ex: Service Urgences -> Unité Traumatologie, Unité Psychiatrie).
- **Entité de Mesure :** La table patient_load agit comme un journal d'activité. Elle stocke le patient_count à une date donnée.
    - *Calcul de dérivation :* L'attribut occupancy_rate est calculé selon la formule :

$$\text{occupancy_rate} = \frac{\text{patient_count}}{\text{service.bed_capacity}} \times 100$$

- **Détachement Temporaire :** La table staff_loan capture le mouvement exceptionnel d'un soignant. Elle référence deux fois la table service (from_service_id et to_service_id), permettant une traçabilité précise pour la facturation interne.

Module 4 : Le Moteur de Planning (F-05, F-07)

Les Contraintes :

- Définir des types de gardes (nuit, jour, astreinte).
- Spécifier les besoins en personnel (min/max) et les certifications requises pour chaque garde.
- Respecter les préférences (souples) et les contraintes (dures) des soignants.

La Solution Architecturale :

- **Le binôme Shift/Requirement :**
  - shift_type définit les propriétés générales (durée, repos).
  - shift définit l'instance de travail dans une unité.
  - shift_required_certification assure que seules les personnes compétentes sont affectées (Contrainte d'intégrité métier).
- **Logique de Préférence :** La table preference utilise le booléen is_hard_constraint. Lors de l'exécution d'un algorithme de génération de planning, une contrainte à true est un filtre bloquant, tandis qu'à false, elle représente un poids à optimiser.

Module 5 : Gouvernance et Flexibilité (F-10)

La Contrainte :

- Les règles légales (ex: maximum de 48h par semaine) ne doivent pas être codées en dur ("hard-coded").

La Solution Architecturale :

- **Table de Configuration (rule) :** Au lieu de créer des colonnes spécifiques dans chaque table, nous utilisons une table de règles générique.
  - rule_type : Identifiant de la règle (ex: 'MAX_WEEKLY_HOURS').
  - value : La valeur numérique associée (ex: 48).
  - valid_from / valid_to : Permet d'anticiper un changement de législation (ex: passage à 35h à une date future) sans interrompre le fonctionnement actuel.

Sommaire des Tables :

I. Bloc Personnel & Compétences (F-01, F-03)

Ce bloc gère l'identité et le "savoir-faire".

1. **staff** : La table maîtresse. Elle contient l'identité physique unique de chaque individu.
2. **role** : Table de référence des métiers (Médecin, IDE, AS).
3. **staff_role** : **Table de liaison (N:N)**. Elle permet à un soignant d'avoir plusieurs rôles (ex: un infirmier qui est aussi formateur).
4. **specialty** : Contient les spécialités médicales.
    - *Contrainte :* Utilise parent_id pour la hiérarchie (Ex: "Réanimation" est enfant de "Anesthésie").
5. **staff_specialty** : **Table de liaison (N:N)** entre le personnel et les spécialités.
6. **certification** : Catalogue des diplômes techniques (ex: AFGSU, PCR).
7. **certification_dependency** : **Table récursive**. Elle définit les prérequis : pour avoir la certif A, il faut posséder la certif B.
8. **staff_certification** : Historique individuel. Elle stocke qui a quoi, depuis quand, et jusqu'à quand (expiration_date).

II. Bloc Contractuel & Social (F-02, F-06, F-07)

Ce bloc définit le cadre légal et les droits de chaque individu.

9. **contract_type** : Définit les règles par défaut du contrat (ex: un CDI a 35h max/semaine).
10. **contract** : L'application réelle du contrat pour un agent.
    - *Contrainte :* Gère la temporalité via start_date et end_date.
11. **absence_type** : Dictionnaire des motifs (Maladie, Congé, Formation).
12. **absence** : Enregistre chaque période d'indisponibilité.
    - *Note :* On y trouve is_planned pour séparer le prévisionnel du réel.
13. **preference** : Stocke les souhaits ou interdits des agents.
    - *Contrainte :* Le champ is_hard_constraint (booléen) est le pivot entre une préférence et une obligation légale/religieuse.

III. Bloc Structure & Flux (F-04, F-08, F-09)

Ici, on définit "où" le travail se passe et comment le personnel circule.

14. **service** : Entité administrative (ex: Cardiologie). Elle possède un manager_id (clé étrangère vers staff).

15. **care_unit** : Entité géographique/opérationnelle (ex: Unité de soins intensifs).
16. **service_status** : **Table d'historique**. Elle permet de savoir si un service était fermé ou en sous-effectif à une date précise (traçabilité).
17. **staff_service_assignment** : Définit le "port d'attache" habituel d'un soignant.
18. **patient_load** : Table de données quotidiennes sur l'occupation des lits pour calculer la charge de travail.
19. **staff_loan** : Gère les transferts temporaires (prêts) d'un agent d'un service A vers un service B.

IV. Bloc Opérationnel : Planning & Gardes (F-05)

C'est le cœur de l'activité quotidienne.

20. **shift_type** : Modèle de garde (ex: "Garde de nuit 12h", "Matin 7h-14h").
21. **shift** : Créneaux à pourvoir dans une unité de soins précise.
    - *Contrainte :* Définit les seuils de sécurité via min_staff et max_staff.
22. **shift_required_certification** : **Table de contrainte métier**. Elle impose que pour travailler sur ce shift, il faut posséder une certification spécifique (ex: Réa obligatoire en SMUR).
23. **shift_assignment** : La table finale qui dit : "M. Martin travaille sur la garde n°452".

V. Bloc Configuration (F-10)

24. **rule** : La table de "paramétrage". Elle permet de stocker des valeurs comme "Durée de repos minimum entre deux gardes" sans les écrire en dur dans le code de l'application.

Toutes ces tables sont liées.

- Pour savoir si un agent peut faire une garde (shift_assignment), le système doit vérifier son contrat (contract), ses compétences (staff_certification), s'il n'est pas en vacances (absence) et s'il n'a pas une contrainte religieuse ce jour-là (preference).

C'est ce qu'on appelle l'**intégrité référentielle**.

Code SQL

Voici le script SQL complet pour générer les 24 tables sous **PostgreSQL**.

```
CREATE TABLE staff (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(20),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE role (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE staff_role (
    staff_id INT REFERENCES staff(id),
    role_id INT REFERENCES role(id),
    PRIMARY KEY (staff_id, role_id)
);

CREATE TABLE specialty (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    parent_id INT REFERENCES specialty(id)
);

CREATE TABLE staff_specialty (
    staff_id INT REFERENCES staff(id),
    specialty_id INT REFERENCES specialty(id),
    PRIMARY KEY (staff_id, specialty_id)
);

CREATE TABLE contract_type (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    max_hours_per_week INT,
    leave_days_per_year INT,
    night_shift_allowed BOOLEAN DEFAULT true
);

CREATE TABLE contract (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES staff(id) NOT NULL,
    contract_type_id INT REFERENCES contract_type(id) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    workload_percent INT DEFAULT 100
);

CREATE TABLE certification (
    id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL
);

CREATE TABLE certification_dependency (
    parent_cert_id INT REFERENCES certification(id),
    required_cert_id INT REFERENCES certification(id),
    PRIMARY KEY (parent_cert_id, required_cert_id)
);

CREATE TABLE staff_certification (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES staff(id) NOT NULL,
    certification_id INT REFERENCES certification(id) NOT NULL,
    obtained_date DATE NOT NULL,
    expiration_date DATE
);

CREATE TABLE service (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    manager_id INT REFERENCES staff(id),
    bed_capacity INT NOT NULL,
    criticality_level INT DEFAULT 1
);

CREATE TABLE care_unit (
    id SERIAL PRIMARY KEY,
    service_id INT REFERENCES service(id) NOT NULL,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE service_status (
    id SERIAL PRIMARY KEY,
    service_id INT REFERENCES service(id) NOT NULL,
    status VARCHAR(50), -- ouvert, fermé, sous-effectif
    start_date DATE NOT NULL,
    end_date DATE
);

CREATE TABLE staff_service_assignment (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES staff(id) NOT NULL,
    service_id INT REFERENCES service(id) NOT NULL,
    start_date DATE NOT NULL,
```

```
    end_date DATE
);

CREATE TABLE shift_type (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    duration_hours INT NOT NULL,
    requires_rest_after BOOLEAN DEFAULT true
);

CREATE TABLE shift (
    id SERIAL PRIMARY KEY,
    care_unit_id INT REFERENCES care_unit(id) NOT NULL,
    shift_type_id INT REFERENCES shift_type(id) NOT NULL,
    start_datetime TIMESTAMP NOT NULL,
    end_datetime TIMESTAMP NOT NULL,
    min_staff INT DEFAULT 1,
    max_staff INT
);

CREATE TABLE shift_required_certification (
    shift_id INT REFERENCES shift(id),
    certification_id INT REFERENCES certification(id),
    PRIMARY KEY (shift_id, certification_id)
);

CREATE TABLE shift_assignment (
    id SERIAL PRIMARY KEY,
    shift_id INT REFERENCES shift(id) NOT NULL,
    staff_id INT REFERENCES staff(id) NOT NULL,
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE absence_type (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    impacts_quota BOOLEAN DEFAULT true
);

CREATE TABLE absence (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES staff(id) NOT NULL,
    absence_type_id INT REFERENCES absence_type(id) NOT NULL,
    start_date DATE NOT NULL,
    expected_end_date DATE NOT NULL,
    actual_end_date DATE,
    is_planned BOOLEAN DEFAULT true
);

CREATE TABLE preference (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES staff(id) NOT NULL,
    type VARCHAR(50), -- préférence ou contrainte
    description TEXT,
    is_hard_constraint BOOLEAN DEFAULT false,
    start_date DATE NOT NULL,
    end_date DATE
);

CREATE TABLE patient_load (
    id SERIAL PRIMARY KEY,
    care_unit_id INT REFERENCES care_unit(id) NOT NULL,
    date DATE NOT NULL,
    patient_count INT NOT NULL,
    occupancy_rate FLOAT
);

CREATE TABLE staff_loan (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES staff(id) NOT NULL,
    from_service_id INT REFERENCES service(id) NOT NULL,
    to_service_id INT REFERENCES service(id) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL
);

CREATE TABLE rule (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    rule_type VARCHAR(50), -- max_hours, rest_time...
    value NUMERIC NOT NULL,
    unit VARCHAR(20), -- hours, days...
    valid_from DATE NOT NULL,
    valid_to DATE
);
```